

Benchmarking Full-Stack ROS 2 Simulation Platforms for Mobile Robots

Ángel Soriano , Román Navarro, Juan Manuel Navarrete, Robert Vasquez, Rafael Martín, Carlos Gascón , and Joel Ramos 

Abstract—Following the official end-of-life of ROS 1 in May 2025, the robotics community is rapidly migrating to ROS 2, necessitating a rigorous re-evaluation of available simulation tools. This letter presents a comparative benchmarking of five widely used simulators: Gazebo (Harmonic), Webots (R2025a), Unity (6.1), NVIDIA Isaac Sim (4.5.0), and Open 3D Engine (O3DE). Using a standardized high-performance workstation and a unified mobile robot model equipped with stereo cameras, 3D LiDAR, and IMU, each platform is evaluated across distinct environments. Quantitative metrics reported include Startup Time, Real-Time Factor (RTF), and computational footprint (CPU, RAM, and GPU usage). Results reveal a clear trade-off between efficiency and visual fidelity: “ROS 2-first” engines like Webots and Gazebo demonstrate competitive startup times and low resource consumption, making them suitable for iterative development and CI/CD pipelines. Conversely, “Graphics-first” engines like Isaac Sim and Unity offer photorealistic rendering suitable for perception training, but at a significantly higher computational cost. Additionally, O3DE is analyzed as a promising native-ROS 2 alternative. To facilitate independent verification and future research, the complete benchmarking suite, including scenes and measurement scripts, is released as an open-source repository.

Index Terms—Benchmarking, Gazebo, mobile robotics, NVIDIA Isaac Sim, O3DE, ROS2, simulators, unity, Webots.

I. INTRODUCTION

THE development of autonomous mobile robots demands rigorous testing and validation cycles that are not only time-consuming and expensive but often impractical to conduct extensively in physical environments. Robotic simulators have emerged as a critical tool to bridge this gap, enabling safe, scalable, and repeatable evaluation of robot behaviors before real-world deployment [1]. Beyond basic testing, modern simulation is pivotal for Continuous Integration and Continuous Deployment (CI/CD) pipelines, enabling automated regression tests, scenario playback, and the generation of synthetic data for training perception algorithms [2].

The robotics software ecosystem has undergone a paradigm shift with the adoption of ROS 2 [3], which offers improved real-time capabilities, modularity, and native multi-robot support. Following the official end-of-life of ROS 1 in May 2025 [4], the community is rapidly migrating to ROS 2 Humble and Jazzy distributions. Consequently, simulation platforms must prioritize

robust and transparent ROS 2 integration. Traditional platforms like the Gazebo family have transitioned to modular architectures (Gazebo Harmonic), while game-engine-based simulators like Unity and NVIDIA Isaac Sim are gaining traction due to their photorealistic rendering capabilities driven by advances in GPU ray-tracing [5], [6].

Given the diversity of simulation platforms available today—ranging from physics-focused engines to visually rich environments—there is a pressing need to evaluate their suitability specifically for ROS 2-based workflows. While resources such as the *best-of-robot-simulators* list [7] provide a high-level overview, there is a lack of rigorous, quantitative benchmarking that compares these engines under identical conditions.

This work targets ROS 2-integrated, full-stack simulation platforms under an end-to-end workflow that includes rendering, sensor generation, ROS 2 middleware communication, and visualization. Standalone physics engines or other non ROS 2 ready simulators commonly used in learning-based control and legged robotics (e.g., MuJoCo, RaiSim, PyBullet, CoppeliaSIM) are intentionally out of scope, as they typically require additional engineering to provide an equivalent ROS 2 sensor pipeline and do not target the same out-of-the-box “sensor-to-control” stack evaluated here.

Accordingly, the benchmark is designed around a perception- and navigation-oriented workload, where rendering, sensor throughput and ROS 2 communication are dominant factors.

This letter addresses this gap by presenting a system-level benchmark of five widely used simulators: **Gazebo (Harmonic)**, **Webots**, **Unity3D**, **NVIDIA Isaac Sim**, and **Open 3D Engine (O3DE)**. Unlike previous studies that often focus on qualitative features [8] or specific agricultural digital twins [9], this work benchmarks the simulators using a standardized hardware setup and a unified robot model (RB-Watcher) equipped with a complex sensor suite (Stereo cameras, 3D LiDAR, IMU).

The main contributions of this work are:

- A quantitative evaluation of performance metrics including Real-Time Factor (RTF), CPU/GPU utilization, RAM footprint, and startup times across different scaling scenarios (1 to 3 robots).
- A qualitative assessment of ROS 2 integration effort, focusing on the ease of setup, sensor fidelity, and native support for standard interfaces like `ros2_control`.
- A reproducible methodology where all scenarios, robot descriptions, and measurement scripts are open-sourced, ensuring that the results can be independently verified.

The findings are intended to support roboticists in selecting the most appropriate simulator based on specific constraints, whether prioritizing photorealism for perception training, computational efficiency for CI pipelines, or scalability for

Received 28 November 2025; accepted 5 March 2026. Date of publication 27 March 2026; date of current version 8 April 2026. This article was recommended for publication by Associate Editor A. Swikir and Editor L. Pallottino upon evaluation of the reviewers’ comments. (Corresponding author: Ángel Soriano.)

The authors are with Robotnik Automation SL, 46980 Valencia, Spain (e-mail: asoriano@robotnik.eu).

Digital Object Identifier 10.1109/LRA.2026.3678463

Simulator	First Release	Architecture	Physics and Rendering Engine	Popularity	Documentation
	Nov '04 (21 years old) Ign Feb '19 (6 years old)	Modular and component-based	Physics engines: ODE, DART, Bullet, Simbody Rendering: OGRE2	2,000+ GitHub stars	https://gazebo.org/docs
 Webots R2025a	Jan '98 (27 years old)	Single-process simulation	Physics engine: ODE Rendering: OpenGL-based	1,000+ GitHub stars	https://cyberbotics.com/doc/guide/index
 unity v6.1	Jun '05 (20 years old)	Proprietary game engine based on C# scripting	Physics engines: NVIDIA PhysX Rendering: HDRP	6,000+ GitHub stars	https://github.com/Unity-Technologies/ROS-TCP-Connector
 NVIDIA Isaac Sim v4.5.0	Dec '19 (6 years old)	NVIDIA Omniverse platform	Physics engines: PhysX Rendering: RTX ray tracing	2,500+ GitHub stars	https://docs.omniverse.nvidia.com/isaacsim/latest/overview.html
 O3DE v2505.0	Jul '21 (4 years old)	Modular open-source engine	Physics engines: PhysX Rendering: Atom	3,000+ GitHub stars	https://www.o3de.org/docs/

Fig. 1. Summary of technical specifications, architecture, and popularity of the evaluated simulators.

multi-robot fleets. Therefore, our comparison is intentionally limited to ROS 2-ready, full-stack platforms commonly used for end-to-end robotics workflows rather than physics engines in isolation.

The remainder of this letter is organized as follows: Section II discusses related work. Section III presents an overview of the selected simulators. Section IV details the experimental setup and hardware specifications. Section V analyzes the quantitative results, followed by a discussion in Section VI. Finally, Section VII concludes the paper.

II. STATE OF THE ART

The evolution of robot simulators has closely paralleled the development of robotic middleware. Early lightweight simulators like Stage provided essential 2D capabilities for the Player/Stage project. With the advent of ROS, Gazebo [1] emerged as the de facto standard, offering full 3D simulation and deep integration with the ROS ecosystem. While platforms like Webots [10] and CoppeliaSim (formerly V-REP) offered robust alternatives with distinct physics engines, Gazebo Classic maintained dominance due to its open-source nature and community support. Comparative studies from this era, such as Sánchez et al. [11] and Pitonakova et al. [12], highlighted the trade-offs between V-REP's feature richness and Gazebo's community-driven ecosystem.

However, the architectural shift to ROS 2 exposed the limitations of legacy monolithic designs. This catalyzed the development of Gazebo (formerly Ignition), built on a modular Entity-Component-System (ECS) architecture to support modern ROS 2 workflows natively [13]. Concurrently, the demand for photorealism in Deep Reinforcement Learning and synthetic data generation brought game-engine-based simulators to the forefront. Unity3D [5] and NVIDIA Isaac Sim [6] leverage high-fidelity rendering pipelines (HDRP, RTX) and GPU acceleration, pushing the boundaries of visual perception tasks. More recently, the Open 3D Engine (O3DE) has entered the robotics domain, offering an open-source, high-fidelity alternative with a native ROS 2 Gem architecture.

Despite this variety, selecting the right tool remains challenging. Recent surveys have comprehensively reviewed emerging trends in realistic simulation [14], yet many comparative studies largely focus on qualitative aspects. For instance, Platt [15] qualitatively compared Unity3D and Gazebo for general robotics tasks. More technical evaluations exist but are often limited in scope; specifically, some benchmarks focus almost exclusively on the low-level accuracy of physics engines (e.g., assessing friction models, contact stability, and solver performance) [16]. While critical for dynamic analysis, these

physics-centric benchmarks do not address the overhead of the entire simulation stack—rendering, sensor generation, and middleware communication—which is vital for deployment validation.

To date, there is a lack of comprehensive studies that rigorously quantify the system-level performance of modern simulators under ROS 2. This work differentiates itself by evaluating the full “sensor-to-control” loop, measuring not just physics stability, but the computational cost (CPU/GPU/RAM) and real-time capabilities (RTF) of the integrated ROS 2 system in multi-robot scenarios.

III. EVALUATED SIMULATORS

This study evaluates five simulation platforms selected based on their popularity, ROS 2 support, and relevance in the current robotics landscape. The selected versions correspond to the stable releases available as of mid-2025, ensuring compatibility with ROS 2 Humble/Jazzy. A visual summary of their key technical characteristics, architecture, and popularity is presented in Fig. 1.

A. Gazebo (Harmonic)

Formerly known as Ignition, Gazebo is the official successor to Gazebo Classic. It features a modular architecture based on the Entity-Component-System (ECS) design pattern, separating physics, rendering, and communication into distinct plugins. It uses *ros_gz_bridge* for ROS 2 communication and supports multiple physics engines (DART, Bullet, ODE), with DART being the default. Rendering is powered by the OGRE2 engine. Despite its “ROS-first” philosophy, it runs as a separate process from the ROS middleware.

B. Webots (R2025a)

Developed by Cyberbotics, Webots is a mature platform (released in 1998) that became open-source in 2018. It uses a highly optimized custom engine based on ODE for physics and an OpenGL-based renderer. For ROS 2, the *webots_ros2* package is employed, which allows the simulator to act largely as a native ROS 2 node, providing a streamlined interface for sensors and actuators directly exposed as topics and services.

C. Unity (6.1)

Unity is a widely used game engine that has found traction in robotics due to its high-fidelity rendering capabilities, supported by the High Definition Render Pipeline (HDRP) and NVIDIA PhysX 4.x. Unlike native robotics simulators, Unity requires an external bridge for ROS 2 integration. The standard *ROS-TCP-Connector* is utilized to serialize messages over TCP between the C# engine environment and the ROS ecosystem. While visually superior, this architecture can introduce latency due to the serialization overhead.

D. NVIDIA Isaac Sim (4.5.0)

Built on the NVIDIA Omniverse platform, Isaac Sim is designed for physically accurate, photorealistic simulation leveraging RTX ray-tracing and GPU acceleration. It integrates PhysX 5 for rigid and soft body dynamics. Crucially, it provides a native ROS 2 bridge, avoiding external TCP connectors and potentially reducing serialization overhead along the sensor-to-ROS pipeline.

E. Open 3D Engine (O3DE v2505.0)

O3DE is an open-source, real-time 3D engine managed by the Linux Foundation. It distinguishes itself with a native ROS 2 Gem, meaning the simulation loop can interact directly with the ROS 2 middleware without intermediate bridges. It uses the Atom renderer for graphics and NVIDIA PhysX for dynamics. Notable for this study is its support for the Robot Gem and the specialized RGL (Robot Gem Lidar) module for GPU-accelerated sensor simulation.

IV. EXPERIMENTAL METHODOLOGY

To ensure fair and reproducible comparisons, all simulators were evaluated using an identical hardware testbed, robot model, and set of scenarios.

A. Hardware and Software Setup

Experiments were conducted on a high-performance workstation equipped with an **Intel Core i7-13700KF** (13th Gen) processor, **32 GB of DDR5 RAM** (4800 MHz), and an **NVIDIA GeForce RTX 4090** (24 GB VRAM) running on driver version 550.163.01. The operating system was **Ubuntu 22.04 LTS** with **ROS 2 Humble** using **FastDDS** [17] as the default middleware implementation (RMW). To minimize variability, all background processes were suspended during benchmarking.

This workstation-class setup reflects the most common execution environment for simulation workloads (lab workstations/servers/cloud) rather than onboard robot compute. It also avoids hardware bottlenecks that could mask simulator-level differences when scaling robot count and sensor workloads.

B. Robot Description

The reference platform is the **RB-Watcher**, a skid-steering mobile robot focused on inspection and surveillance operations. The simulated model replicates the physical robot's kinematics and sensor suite. The sensor configuration used for benchmarking includes:

- **1 × 3D LiDAR:** configured with 28,800 points per scan at 10Hz.
- **2 × RGB-D Cameras:** Camera 1 at 1280 × 720 resolution (30 FPS) and Camera 2 at 1920 × 1080 resolution (30 FPS).
- **1 × IMU:** streaming at 200Hz.

1) **Kinematic Model:** Although the robot uses skid-steering, for control purposes the kinematic model is approximated to a unicycle form. Under no-slip assumptions, the body-frame linear (v) and angular (ω) velocities relate to the right (ω_R) and left (ω_L) wheel angular velocities as:

$$v = \frac{r}{2} (\omega_R + \omega_L), \quad \omega = \frac{r}{L} (\omega_R - \omega_L) \quad (1)$$

where r is the wheel radius and L is the wheel separation. The planar pose (x, y, θ) evolves as:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega \quad (2)$$

Accurate reproduction of ground contact, friction, and inertial properties is critical for the simulators to respect this model without excessive drift. Therefore, the physics configuration (time step, solver iterations, contact parameters) was explicitly pinned per simulator configuration.

C. Test Scenarios and Protocol

The simulators were evaluated in two distinct environments:

- 1) **Empty World:** A plain plane with no obstacles, serving as a baseline to measure the minimum overhead of the physics and sensor generation engines.
- 2) **Structured Indoor World:** A complex warehouse-like environment with walls and static obstacles, testing the rendering pipeline and collision checking performance.

Experiments were scaled by varying the number of active robots in the scene (**1, 2, and 3 robots**). To simulate a realistic load characteristic of autonomous navigation tasks [18], an instance of **RViz2** was running concurrently, subscribing to all camera feeds and LiDAR topics. Each configuration (Scenario × Robot Count) was executed **5 times** to ensure statistical significance.

D. Evaluation Metrics

The performance is quantified using the following metrics:

- **Startup Time (s):** Time elapsed from the launch command until the first camera frame is received on the ROS topic.
- **Real-Time Factor (RTF):** The ratio between simulated time and wall-clock time ($RTF = t_{sim}/t_{real}$). A value of 1.0 indicates real-time performance; values < 1.0 indicate the simulator is slower than reality.
- **Resource Usage:** Average CPU utilization (normalized percentage), RAM usage (MB), and GPU load (%) recorded during a steady-state period of 60 seconds.

In addition to the GUI-based configuration (simulator + RViz2), this work evaluated a headless execution mode to reflect common CI and large-scale learning setups. In this mode, the simulator runs without an interactive GUI and RViz2 is disabled. All robot instances still publish the same ROS 2 sensor topics (cameras, LiDAR, IMU) at the configured rates, and metrics are recorded using the same procedure and time windows as in the GUI experiments. The results are shown in Section V-F.

E. Reproducibility and Data Availability

To facilitate independent verification and further research, the complete benchmarking suite has been released as an open-source repository. This includes the containerized environments, robot descriptions (URDF/XACRO), simulation worlds, and the automated scripts used to capture the metrics presented in this study. The source code and documentation are available at RobotnikAutomation/robotnik_sim_benchmark and permanently archived on Zenodo (DOI: 10.5281/zenodo.18879495).

V. EXPERIMENTAL RESULTS

This section presents the quantitative performance of the five simulators. The metrics are grouped into initialization latency, real-time capability, and computational resource demand.

A. Initialization Latency

Startup time is a critical metric for development workflows, particularly in CI/CD pipelines where simulations are restarted frequently. Fig. 2 illustrates the time required to reach a “ready” state.

Webots and **Unity** demonstrated the fastest startup times, averaging under 5.2 seconds in both empty and structured environments. This low latency facilitates rapid iterative testing.

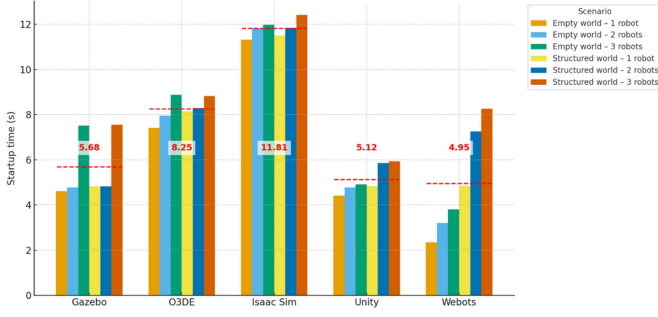


Fig. 2. Average startup time per scenario (Empty vs. Structured) and number of robots. Lower is better.

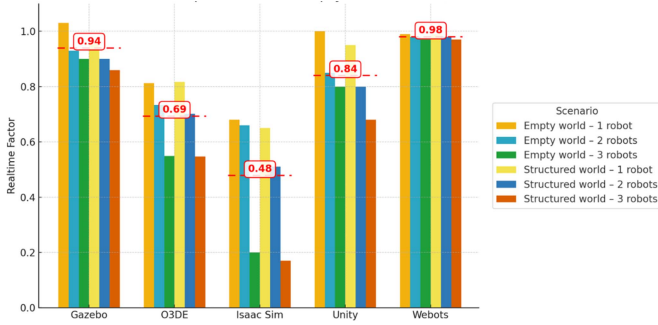


Fig. 3. Real-Time Factor (RTF) evolution. The dashed line represents RTF=1 (ideal real-time).

Gazebo follows closely with an average of 5.68 seconds, showing only a marginal increase when adding robots. Conversely, **Isaac Sim** exhibited the highest latency, averaging 11.8 seconds, likely due to the overhead of loading the Omniverse framework and RTX rendering pipeline. **O3DE** also showed higher load times (approx. 8.25 s), reflecting the initialization cost of its engine modules.

B. Real-Time Performance

The Real-Time Factor (RTF) measures the simulator's ability to keep pace with wall-clock time, which is essential for sensor data validity.

As shown in Fig. 3, **Webots** achieved the most stable performance, maintaining an RTF ≈ 0.98 even with three robots in a structured environment. **Gazebo** maintained real-time speeds (RTF ≈ 0.94) for 1 and 2 robots but showed a performance drop when scaling to 3 robots. **Unity** started strong (RTF 0.84) but degraded linearly with scene complexity. **Isaac Sim** struggled to maintain real-time performance on this specific hardware setup, with RTF values dropping to 0.48 in complex scenarios, suggesting that its high-fidelity physics and rendering require more computational headroom than available or further optimization.

C. Computational Footprint: CPU and RAM

The resource usage analysis reveals distinct resource management strategies for each engine, as detailed below.

CPU Usage: Fig. 4 illustrates the normalized CPU load. **Webots**, **Unity**, and **O3DE** proved to be extremely efficient, keeping usage consistently below 10% per core. **Gazebo** showed

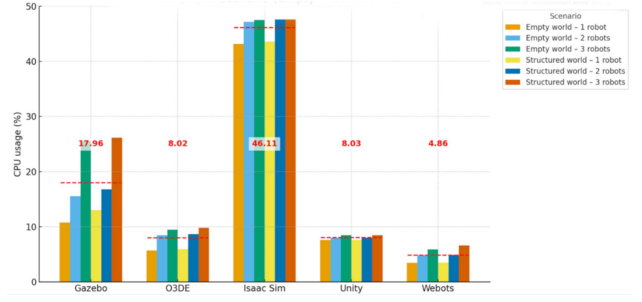


Fig. 4. Average normalized CPU usage per core. Isaac Sim shows a significantly higher demand compared to the other engines.

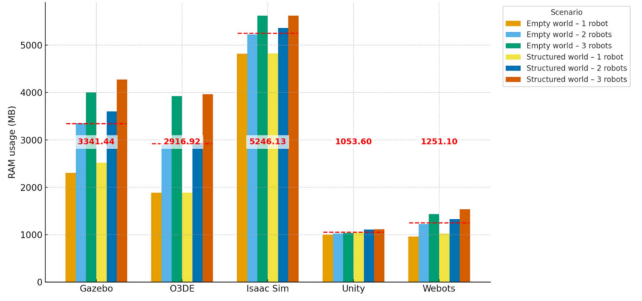


Fig. 5. Average RAM usage (RSS). Unity and Webots demonstrate the lowest memory footprint.

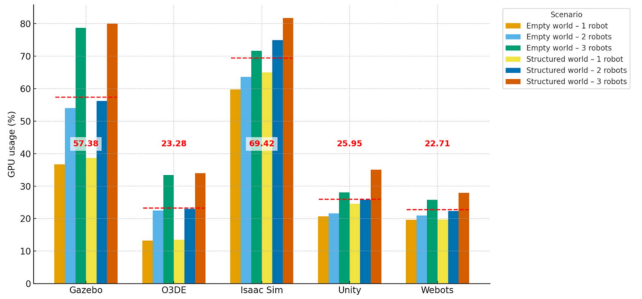


Fig. 6. Average GPU utilization. High-fidelity simulators like Isaac Sim impose a significantly higher load on the graphics card.

moderate usage ($\sim 18\%$). In contrast, **Isaac Sim** was significantly more compute-intensive ($\sim 46\%$), likely due to the combined overhead of its rendering/sensor simulation pipeline and runtime layers required to interface with ROS 2 while maintaining scene updates and physics at scale.

RAM Usage: As shown in Fig. 5, **Unity** and **Webots** are the most lightweight platforms, consuming approximately 1 GB of RAM, making them suitable for constrained hardware. **O3DE** and **Gazebo** occupied the middle ground (~ 3 GB). **Isaac Sim** required the most memory, exceeding 5GB, which is consistent with its comprehensive feature set and asset-heavy environment.

D. GPU Utilization

Given the focus on perception (cameras and LiDAR), GPU load is a limiting factor. Fig. 6 presents the results.

Isaac Sim and **Gazebo** made the most intensive use of the GPU (approx. 70% and 57% respectively), driven by their rendering pipelines (RTX and OGRE2) and GPU-accelerated sensor generation. **Unity**, despite its graphical fidelity, managed resources efficiently ($\sim 26\%$), similar to **O3DE** and **Webots**

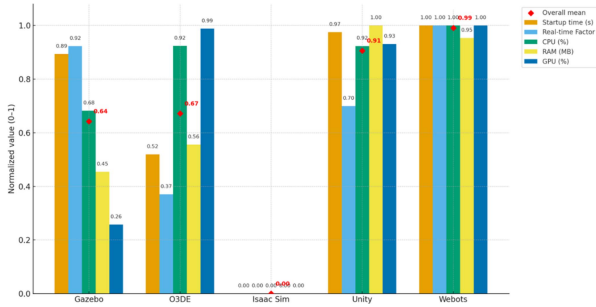


Fig. 7. Normalized metrics comparison (0-1 scale). Values closer to 1 indicate better performance (higher RTF) or lower resource consumption (Time, CPU, RAM, GPU).

TABLE I
NORMALIZED HEADLESS BENEFIT OVER GUI (CLIPPED AT 0)

Sim.	Start. (%)	RTF (%)	CPU (%)	GPU RAM (%)	Avg. (%)
Gazebo	0.0	0.2	31.7	37.5	17.4
O3DE	0.0	13.7	16.7	13.2	10.9
Isaac Sim	20.7	21.1	0.0	5.5	11.8
Unity	40.2	0.0	0.0	11.2	12.9
Webots	7.1	0.0	29.5	0.0	9.2

(~23%), suggesting optimized render loops or simpler shader complexity in the default configurations.

E. Performance Summary

To provide a holistic view, Fig. 7 presents a normalized comparison of all metrics. This visualization reinforces the trade-off between fidelity and efficiency: lighter simulators (Webots, Unity) excel in RTF and startup times, while feature-rich platforms (Isaac Sim, Gazebo) demand more resources to deliver their advanced capabilities.

F. Headless Execution Performance

To better reflect large-scale robotics workflows, Table I summarizes the headless impact (no GUI, no RViz2) averaged over the same scenarios and robot counts. Each entry reports the *relative improvement* of headless over GUI, clipped at 0 (Startup/CPU/GPU RAM: (GUI – Headless)/GUI; RTF: (Headless – GUI)/GUI), and Avg. is the unweighted mean; overall, Gazebo shows the largest average gain, driven by CPU and GPU-memory savings. Here, 0% indicates no improvement under our clipping, i.e., headless did not yield a consistent advantage for that metric under the aggregated results.

VI. DISCUSSION

The results of this study suggest that there is no “one-size-fits-all” simulator for ROS 2. Instead, the choice depends heavily on the specific requirements of the robotic application. Three distinct categories are identified based on the trade-off between integration effort, computational efficiency, and visual fidelity.

A. The “ROS 2-First” Approach: Gazebo and Webots

Gazebo (Harmonic) and Webots represent the most balanced options for general-purpose robotics R&D. Their primary strength lies in their low barrier to entry and efficiency.

- **Efficiency:** Both engines maintain high Real-Time Factors (RTF ≈ 0.94 – 0.98) and low resource footprints, both engines maintain high Real-Time Factors and moderate

resource footprints, and the headless experiments further confirm their suitability for CI pipelines and large-scale automated testing, where simulations must run reliably without interactive visualization.

- **Integration:** Being deeply rooted in the robotics community, their ROS 2 interfaces (*ros_gz_bridge* and *webots_ros2*) are mature.
- **Limitation:** Their rendering engines (OGRE2 and OpenGL), while functional, lag behind the photorealism required for training modern vision-based AI models.

B. The “Graphics-First” Approach: Isaac Sim and Unity

NVIDIA Isaac Sim and Unity prioritize visual fidelity, leveraging game-engine technologies (RTX, HDRP) to produce photorealistic synthetic data.

- **Fidelity:** These simulators are the superior choice for perception tasks, such as training object detection or semantic segmentation networks using domain randomization techniques [19], where lighting and texture realism are critical to minimize the “sim-to-real” gap.
- **Cost:** This fidelity comes at a steep computational price. Isaac Sim, in particular, demands significant GPU and RAM resources, making it less suitable for low-end workstations or standard CI runners.
- **Connectivity:** While Isaac Sim offers a high-performance native bridge, Unity relies on a TCP connector which can introduce latency bottlenecks in high-frequency control loops.

C. The Native Engine Approach: O3DE

Open 3D Engine (O3DE) occupies a promising middle ground. By integrating ROS 2 directly into the engine loop via Gems, it avoids some of the bridging overheads. Its performance is competitive, and its visual quality is high (Atom renderer). However, as a newer entrant to the robotics domain, its community support and documentation are currently less extensive than Gazebo’s, implying a steeper learning curve for setup and debugging.

D. Headless Implications

With visualization removed, the results show that performance differences are largely driven by each simulator’s sensor-generation pipeline and ROS 2 integration path, rather than GUI rendering overhead; headless primarily benefits startup and RTF in some platforms, while resource gains depend on how sensors and middleware are implemented. In our setup, Gazebo exhibits the largest overall headless benefit, mainly through CPU and GPU-memory reductions.

VII. CONCLUSION

This letter presented a comparative benchmarking of five major simulators—Gazebo, Webots, Unity, NVIDIA Isaac Sim, and O3DE—within a standardized ROS 2 Humble workflow. Experiments conducted on a unified hardware testbed using a skid-steering-drive robot equipped with diverse sensors revealed a clear inverse correlation between visual fidelity and real-time performance.

While skid-steering behavior is sensitive to ground contact and friction modeling, the current benchmark does not target

contact-rich, high-DoF control (e.g., legged locomotion or manipulation). Extending the framework to such platforms is part of our planned future work.

”ROS 2-first” engines like Gazebo and Webots demonstrated superior efficiency, maintaining real-time performance with lower resource consumption, making them ideal for algorithmic development and continuous integration (CI) pipelines. In contrast, ”Graphics-first” engines like Isaac Sim and Unity provided the high-fidelity rendering required for perception training but at a significantly higher computational cost, necessitating powerful GPU hardware. O3DE was identified as a promising alternative, offering native integration and a balance between fidelity and performance, though within a newer ecosystem.

Beyond these specific quantitative findings, a primary contribution of this work is the open-source benchmarking framework itself. Given the rapid evolution of rendering engines and middleware, static comparisons quickly become obsolete. This repository is designed as a living tool for the community, enabling the continuous evaluation of future simulator releases and the tracking of computational requirements over time. Its modular architecture allows researchers to easily extend the study by adding new simulators, modifying sensor payloads (e.g., evaluating the impact of increasing LiDAR resolution or camera count), or designing stress tests for specific hardware configurations. By facilitating reproducible and extensible performance tracking, this framework aims to provide a standardized baseline for assessing how software updates impact simulation efficiency in an increasingly demanding technological landscape.

Future work will include benchmarking across additional workstation hardware tiers (e.g., mid-range lab/classroom PCs, standard GPU servers or even embedded configurations like Jetsons or similar) to quantify the sensitivity of the reported trade-offs to compute budgets.

Additionally, the objective is to expand the sensor suite to include realistic noise models and evaluate the ”sim-to-real” transferability of control policies trained across these diverse environments.

REFERENCES

- [1] N. Koenig and A. Howard, ”Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2004, pp. 2149–2154.
- [2] S. Salimpour, J. Peña-Queralta, D. Paez-Granados, J. Heikkonen, and T. Westerlund, ”Sim-to-real transfer for mobile robots with reinforcement learning: From NVIDIA Isaac Sim to Gazebo and real ROS 2 robots,” 2025, *arXiv:2501.02902*.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, ”Robot operating system 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, 2022, Art. no. eabm6074.
- [4] O. Robotics, ”ROS1 end of life announcement,” 2023. [Online]. Available: <https://discourse.ros.org/t/ros-1-noetic-end-of-life-announcement/32321>
- [5] A. Juliani, V.-P. Berges, and E. Teng, ”Unity: A general platform for intelligent agents,” 2018, *arXiv:1809.02627*.
- [6] V. Makovychuk and L. Wawrzyniak, ”Isaac Gym: High performance GPU-based physics simulation for robot learning,” 2021, *arXiv:2108.10470*.
- [7] K. N. McGuire, ”Best of robot simulators,” 2024. [Online]. Available: <https://github.com/knmcguire/best-of-robot-simulators>
- [8] C. A. Dimmig, G. Silano, and K. McGuire, ”Survey of simulators for aerial robots,” 2024, *arXiv:2311.02296*.
- [9] A. Rana, A. Petitti, A. Ugenti, R. Galati, G. Reina, and A. Milella, ”Toward digital twin of off-road vehicles using robot simulation frameworks,” *IEEE Access*, vol. 12, pp. 178047–178061, 2024.
- [10] O. Michel, ”Webots: Professional mobile robot simulation,” *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.
- [11] D. Sánchez and C. Rodríguez, ”A comparison between Gazebo and V-REP for robotic simulation in education and research,” *J. Intell. Robot. Syst.*, vol. 81, pp. 61–72, 2016.
- [12] L. Pitonakova, M. Giuliani, A. Pipe, and A. Winfield, ”Feature and performance comparison of the V-REP, Gazebo and ARGoS robot simulators,” in *Proc. Annu. Conf. Towards Auton. Robot. Syst.*, 2018, pp. 357–368.
- [13] F. P. Audonnet, A. Hamilton, and G. Aragon-Camarasa, ”A systematic comparison of simulation software for robotic arm manipulation using ROS 2,” 2022, *arXiv:2204.06433*.
- [14] S. M. Kargar, B. Yordanov, C. Harvey, and A. Asadipour, ”Emerging trends in realistic robotic simulations: A comprehensive systematic literature review,” *IEEE Access*, vol. 12, pp. 191264–191287, 2024.
- [15] J. T. Platt, ”Comparative analysis of ROS-unity3D and ROS-Gazebo for mobile ground robot simulation,” Master’s thesis, Univ. Alabama, Tuscaloosa, AL, 2022. [Online]. Available: <https://ir.ua.edu/handle/123456789/8507>
- [16] K. Yaswanth, ”Simulation benchmark: Physics engine comparison for robotics,” 2024. Accessed: May 15, 2025. [Online]. Available: https://github.com/yaswanth1701/simulation_benchmark
- [17] eProsima, Fast DDS documentation: High performance and low latency, 2024. [Online]. Available: <https://fast-dds.docs.eprosima.com/>
- [18] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, ”The marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 2718–2725.
- [19] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, ”Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 23–30.